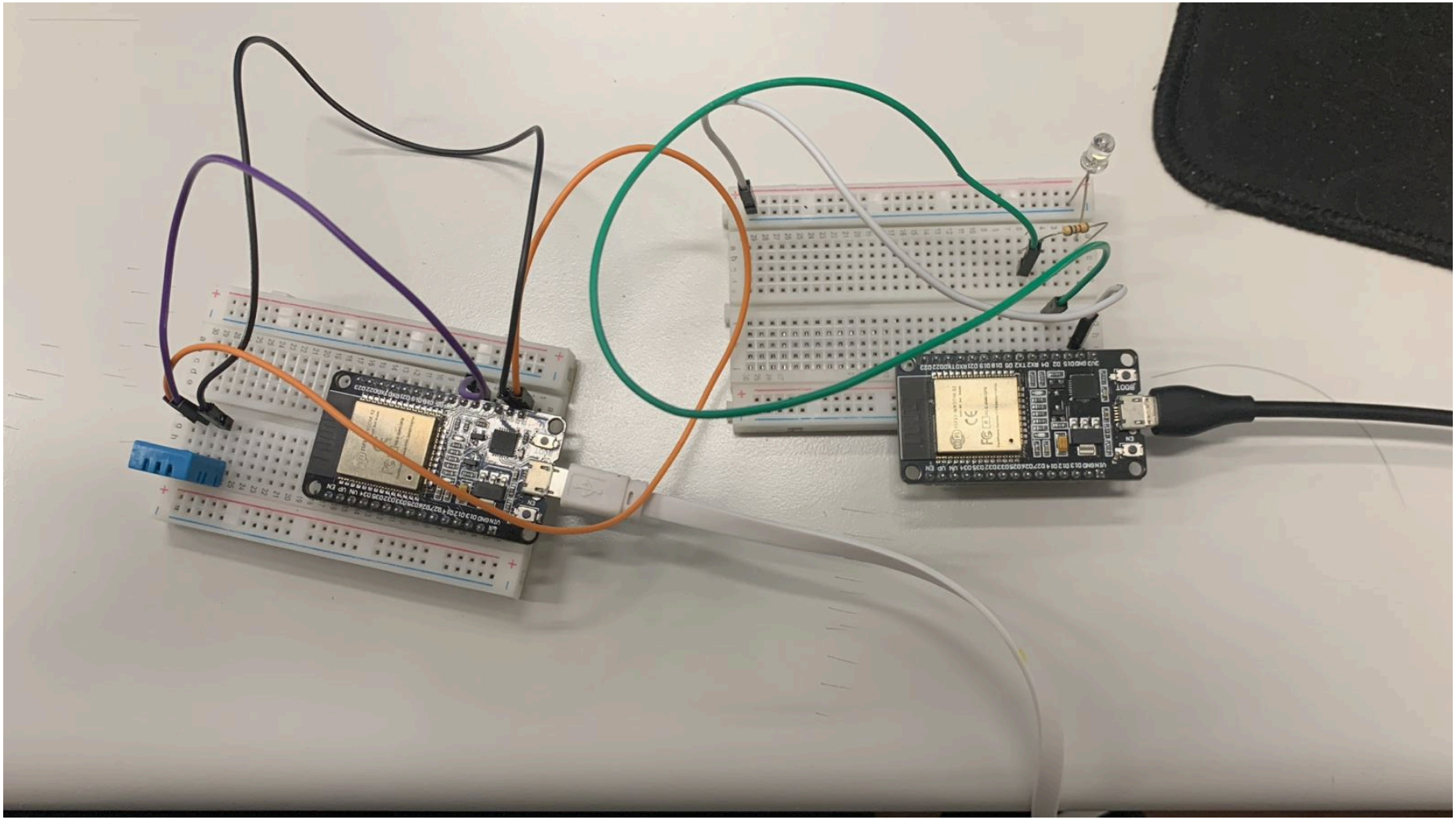
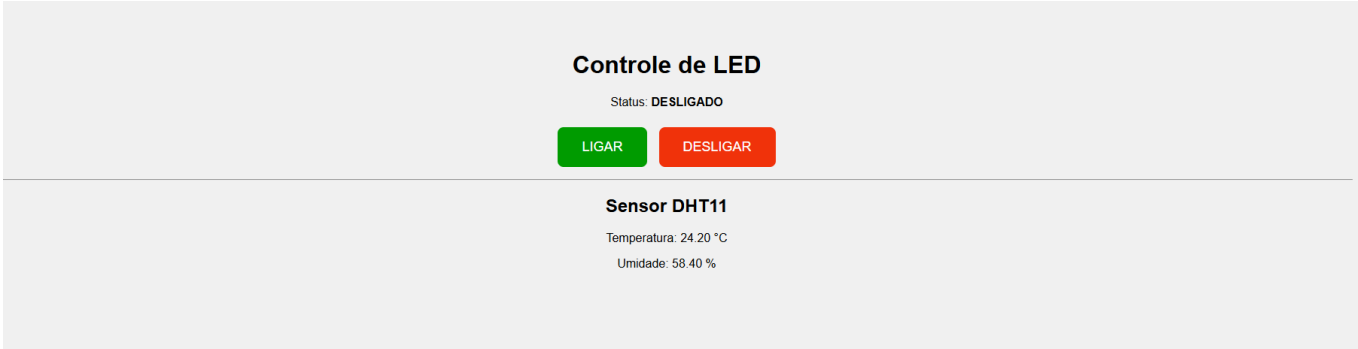
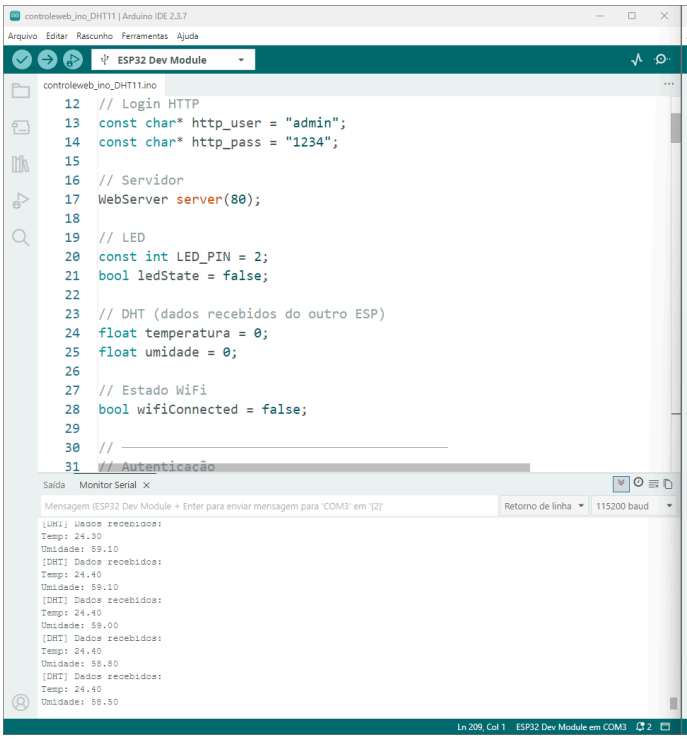
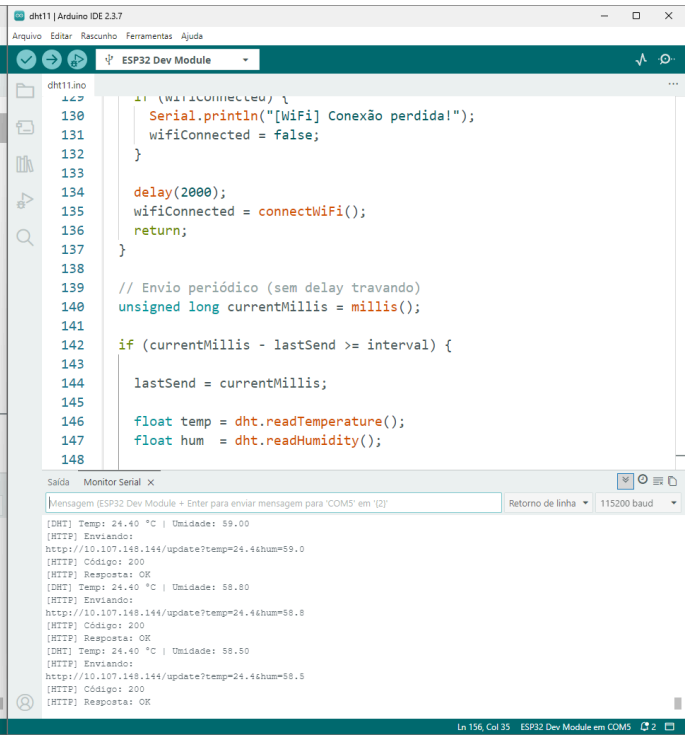
HTTP - LAN

Trabalharemos com dois esp32









ESP32 - SERVIDOR

```
// ──────────────────────────────────────────
// ESP32 Web Server — LED + DHT11 Dashboard
// ──────────────────────────────────────────

#include <WiFi.h>
#include <WebServer.h>

// WiFi
const char* ssid     = "nome da rede";
const char* password = "senha";

// Login HTTP
const char* http_user = "admin";
const char* http_pass = "1234";

// Servidor
WebServer server(80);

// LED
const int LED_PIN = 2;
bool ledState = false;

// DHT (dados recebidos do outro ESP)
float temperatura = 0;
float umidade = 0;

// Estado WiFi
bool wifiConnected = false;

// ──────────────────────────────────────────
// Autenticação
// ──────────────────────────────────────────
bool isAuthenticated() {

  if (!server.authenticate(http_user, http_pass)) {
    server.requestAuthentication();
    return false;
  }

  return true;
}

// ──────────────────────────────────────────
// HTML
// ──────────────────────────────────────────
String buildPage() {

  String html = "<!DOCTYPE html><html><head>";
  html += "<meta charset='utf-8'>";
  html += "<meta name='viewport' content='width=device-width'>";
  html += "<title>ESP32 Dashboard</title>";

  html += "<style>";
  html += "body{font-family:sans-serif;text-align:center;padding:40px;background:#f4f4f4}";
  html += "button{padding:16px 32px;font-size:18px;border:none;border-radius:8px;margin:8px}";
  html += ".on{background:#009d00;color:#fff}";
  html += ".off{background:#F4320B;color:#fff}";
  html += "</style>";

  html += "</head><body>";

```

```
  html += "<h1>Controle de LED</h1>";

  html += "<p>Status: <strong>";
  html += (ledState ? "LIGADO" : "DESLIGADO");
  html += "</strong></p>";

  html += "<a href='/on'><button class='on'>LIGAR</button></a>";
  html += "<a href='/off'><button class='off'>DESLIGAR</button></a>";

  html += "<hr>";

  html += "<h2>Sensor DHT11</h2>";

  html += "<p>Temperatura: ";
  html += String(temperatura);
  html += " ºC</p>";

  html += "<p>Umidade: ";
  html += String(umidade);
  html += " %</p>";

  html += "</body></html>";

  return html;
}

// ────────────────────────────────────────
// WiFi
// ────────────────────────────────────────
bool connectWiFi() {

  Serial.println("\n[WiFi] Resetando interface...");

  WiFi.disconnect(true);
  delay(1000);

  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);

  Serial.println("[WiFi] Conectando...");
  Serial.print("[WiFi] SSID: ");
  Serial.println(ssid);

  int tentativas = 0;

  while (WiFi.status() != WL_CONNECTED && tentativas < 20) {
    delay(500);
    Serial.print(".");
    tentativas++;
  }

  if (WiFi.status() == WL_CONNECTED) {

    Serial.println("\n[WiFi] Conectado!");
    Serial.print("[WiFi] IP: ");
    Serial.println(WiFi.localIP());

    return true;

  } else {

    Serial.println("\n[WiFi] Falha na conexão.");
    return false;

  }
```

```
  }
}

// ────────────────────────────────────────
// Rotas
// ────────────────────────────────────────
void setupRoutes() {

  // Página principal
  server.on("/", []() {

    if (!isAuthenticated()) return;

    server.send(200, "text/html", buildPage());
  });

  // Ligar LED
  server.on("/on", []() {

    if (!isAuthenticated()) return;

    ledState = true;
    digitalWrite(LED_PIN, HIGH);

    Serial.println("[LED] Ligado");

    server.sendHeader("Location", "/");
    server.send(303);
  });

  // Desligar LED
  server.on("/off", []() {

    if (!isAuthenticated()) return;

    ledState = false;
    digitalWrite(LED_PIN, LOW);

    Serial.println("[LED] Desligado");

    server.sendHeader("Location", "/");
    server.send(303);
  });

  // Receber dados do ESP DHT
  server.on("/update", []() {

    // SEM autenticação aqui

    if (server.hasArg("temp")) {
      temperatura = server.arg("temp").toFloat();
    }

    if (server.hasArg("hum")) {
      umidade = server.arg("hum").toFloat();
    }

    Serial.println("[DHT] Dados recebidos:");
    Serial.print("Temp: ");
    Serial.println(temperatura);
    Serial.print("Umidade: ");
    Serial.println(umidade);

    server.send(200, "text/plain", "OK");
  });
```

```
}

// ────────────────────────────────────────
// Setup
// ────────────────────────────────────────
void setup() {

  Serial.begin(115200);
  delay(1000);

  Serial.println("\n[BOOT] Inicializando ESP32...");

  pinMode(LED_PIN, OUTPUT);
  digitalWrite(LED_PIN, LOW);

  wifiConnected = connectWiFi();

  setupRoutes();

  if (wifiConnected) {

    server.begin();
    Serial.println("[HTTP] Servidor iniciado!");

  } else {

    Serial.println("[HTTP] Servidor NÃO iniciado (sem WiFi)");
  }
}

// ────────────────────────────────────────
// Loop
// ────────────────────────────────────────
void loop() {

  if (WiFi.status() != WL_CONNECTED) {

    if (wifiConnected) {
      Serial.println("[WiFi] Conexão perdida!");
      wifiConnected = false;
    }

    delay(2000);
    wifiConnected = connectWiFi();

    if (wifiConnected) {
      server.begin();
      Serial.println("[HTTP] Servidor reiniciado!");
    }

    return;
  }

  server.handleClient();
}
```

ESP32 - SENSOR DHT11

```
// ────────────────────────────────────────────
// ESP32 Cliente — DHT11 → Envio HTTP
// ────────────────────────────────────────────

#include <WiFi.h>
#include <HTTPClient.h>
#include "DHT.h"

// DHT
#define DHTPIN 4
#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);

// WiFi
const char* ssid     = "nome da rede";
const char* password = "senha";

// Servidor (ESP32 #1)
const char* serverIP = "10.107.148.144";

// Controle
unsigned long lastSend = 0;
const long interval = 5000; // 5 segundos

bool wifiConnected = false;

// ────────────────────────────────────────────
// WiFi
// ────────────────────────────────────────────
bool connectWiFi() {

  Serial.println("\n[WiFi] Resetando interface...");

  WiFi.disconnect(true);
  delay(1000);

  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);

  Serial.println("[WiFi] Conectando...");
```

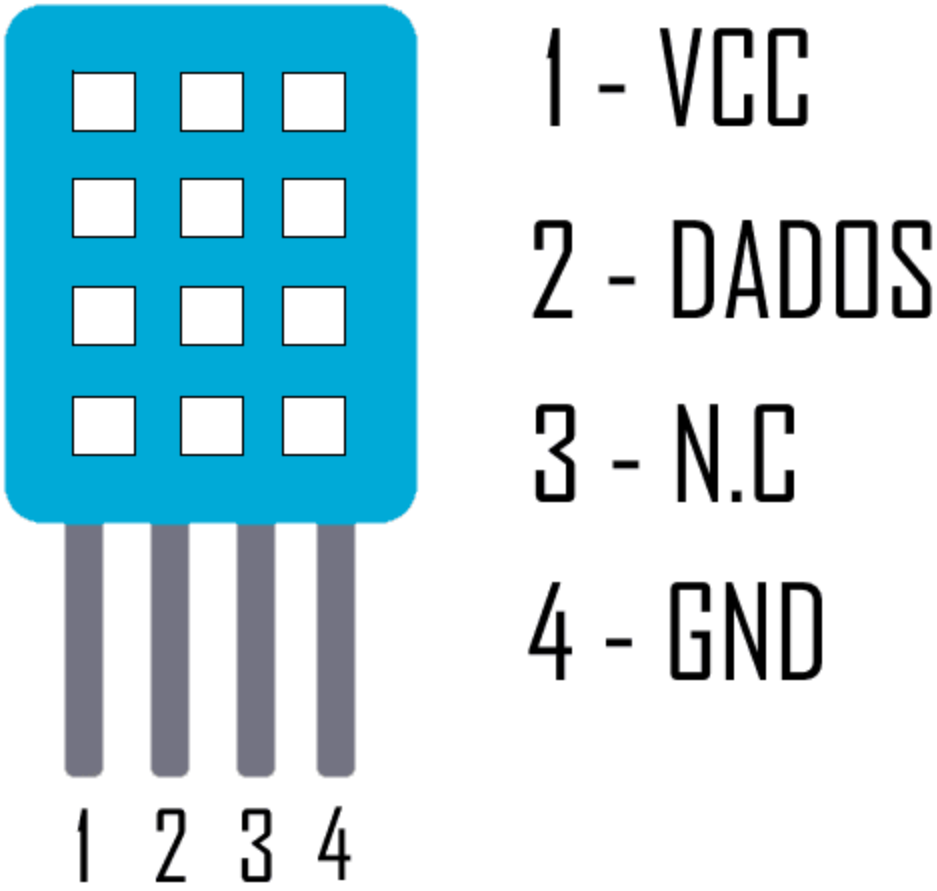
```
42    Serial.print("[WiFi] SSID: ");
43    Serial.println(ssid);
44
45    int tentativas = 0;
46
47    while (WiFi.status() != WL_CONNECTED && tentativas < 20) {
48      delay(500);
49      Serial.print(".");
50      tentativas++;
51    }
52
53    if (WiFi.status() == WL_CONNECTED) {
54
55      Serial.println("\n[WiFi] Conectado!");
56      Serial.print("[WiFi] IP: ");
57      Serial.println(WiFi.localIP());
58
59      return true;
60
61    } else {
62
63      Serial.println("\n[WiFi] Falha na conexão.");
64      return false;
65
66    }
67  }
68
69  // ────────────────────────────────────────
70  // Envio HTTP
71  // ────────────────────────────────────────
72  void sendData(float temp, float hum) {
73
74    HTTPClient http;
75
76    String url = "http://" + String(serverIP) +
77                 "/update?temp=" + String(temp, 1) +
78                 "&hum=" + String(hum, 1);
79
80    Serial.println("[HTTP] Enviando:");
81    Serial.println(url);
82
83    http.begin(url);
84    http.setTimeout(3000); // 3 segundos
85
```

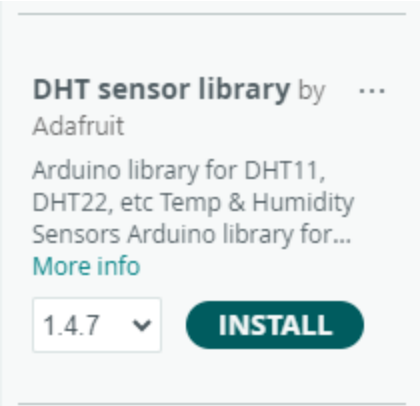
```
86    int httpCode = http.GET();
87
88    if (httpCode > 0) {
89
90      Serial.print("[HTTP] Código: ");
91      Serial.println(httpCode);
92
93      String payload = http.getString();
94      Serial.print("[HTTP] Resposta: ");
95      Serial.println(payload);
96
97    } else {
98
99      Serial.print("[HTTP] Erro: ");
100     Serial.println(httpCode);
101   }
102
103   http.end();
104 }
105
106 // ────────────────────────────────────────
107 // Setup
108 // ────────────────────────────────────────
109 void setup() {
110
111   Serial.begin(115200);
112   delay(1000);
113
114   Serial.println("\n[BOOT] Inicializando DHT Client...");
115
116   dht.begin();
117
118   wifiConnected = connectWiFi();
119 }
120
121 // ────────────────────────────────────────
122 // Loop
123 // ────────────────────────────────────────
124 void loop() {
125
126   // Reconexão WiFi
127   if (WiFi.status() != WL_CONNECTED) {
128
129     if (wifiConnected) {
130       Serial.println("[WiFi] Conexão perdida!");
```

```
      Serial.println("[WiFi] Conexão perdida!");
      wifiConnected = false;
    }

    delay(2000);
    wifiConnected = connectWiFi();
    return;
  }

  // Envio periódico (sem delay travando)
  unsigned long currentMillis = millis();

  if (currentMillis - lastSend >= interval) {

    lastSend = currentMillis;

    float temp = dht.readTemperature();
    float hum  = dht.readHumidity();

    if (isnan(temp) || isnan(hum)) {
      Serial.println("[DHT] Falha na leitura");
      return;
    }

    Serial.print("[DHT] Temp: ");
    Serial.print(temp);
    Serial.print(" °C | Umidade: ");
    Serial.println(hum);

    sendData(temp, hum);
  }
}
```

Repositório

Atenção


1 - VCC
2 - DADOS
3 - N.C
4 - GND
1 2 3 4


DHT sensor library by Adafruit
Arduino library for DHT11, DHT22, etc Temp & Humidity Sensors Arduino library for...
More info
1.4.7
INSTALL